



ZenVoice AI

禪音智友

佛經 Agent 語音助手 App



開口就能聽佛經，看著螢幕就能讀佛經。

Presented by

SD Wu & Clay Huang



CONTENTS

目錄

序

前言

緣起與構想

01

第一章

實用價值暨實際 DEMO 體驗

02

第二章

導入或商業化計劃

03

第三章

技術實現與工具運用

04

第四章

資訊安全

結

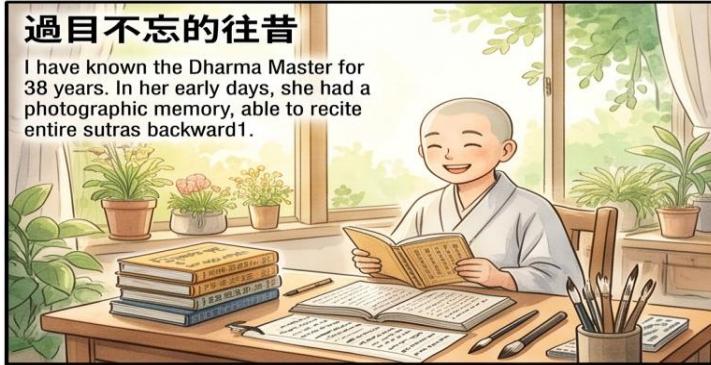
總結

回顧與展望

目錄

過目不忘的往昔

I have known the Dharma Master for 38 years. In her early days, she had a photographic memory, able to recite entire sutras backward¹.



青光眼的侵襲

But as the Master entered her eighties, in the last three years, her vision has severely deteriorated due to glaucoma, leaving only peripheral light perception¹.



承載 30 年修行的白色小機

One day she asked me to repair this malfunctioning machine. I was shocked to find that the 1,200 MP3 files inside contained thirty years of her Buddhist studies and spiritual sustenance¹.



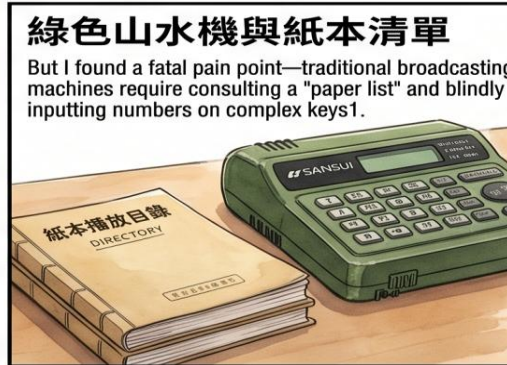
開發者的自動化努力

To expand the content, I learned Vibe coding to write Python, automatically organizing over 2,000 files¹.



綠色山水機與紙本清單

But I found a fatal pain point—traditional broadcasting machines require consulting a "paper list" and blindly inputting numbers on complex keys¹.



無法跨越的操作高牆

For the blind, this is a maze. She is terrified of pressing the wrong key and mixing up the settings, only using these two buttons for a year and a half¹.



一聲呼喚，佛音隨行

"I want to listen to the sutra about 'Avalokitesvara Bodhisattva'..."⁵ "Master, you must mean the 'Heart Sutra'. Playing it now."⁵



AI 語音助理的誕生

Software automation wasn't enough; we had to break down the hardware wall! The voice² was born⁴.



解放雙手的修行

The system can seamlessly resume yesterday's progress. Visually impaired people outdoors only need to tap earbuds to wind play, completely hands-free⁶.



點亮心燈的慈悲科技

Using AI to instantly cleanse collation codes and generate pure audio⁶. We allow those who cannot see to hear the Dharma, allowing wisdom anapragate without hindrance⁴.

禪音智友
(ZenVoice AI)⁴



緣起 | 當眼睛闔上，修行的心不能斷

一位青光眼失明法師的聽經難題，與一位義工的深夜接力



法師的困境



高齡漸進失明

80 歲法師因青光眼，無法再以視覺讀經



只能用耳朵聽經

僅能透過語音轉文字、聽 MP3 聞法



困在數字檔案迷宮

面對 001.mp3、002.mp3... 毫無語意的編號

80 歲 · 全盲

僅靠四位數字型 MP3 播放器



×



義工的極限



純手工轉檔

逐一下載、清理、轉錄每一個音檔



深夜勞力透支

耗時整整三個月的接力整理



冰冷機器音

檔名繁雜難維護，聽感生硬無溫度

2,000+ 個音檔

耗時 3 個月純手工整理

繁雜的檔案管理與冰冷的機器音，不僅讓照顧者身心透支，更剝奪了法師自主聽經的尊嚴。

→ 這正是 ZenVoice AI 的起點

構想 | 從勞力到智力，讓 AI Agent 接管前段辛勞

三段演進：從純人工，到程式輔助，再躍遷到 AI Agent 全自動化

1.0

苦勞摸索

(人工階段)



純人工下載、清理、轉錄與校對，
流程繁瑣、維護極為困難。

耗時 3 個月



2.0

程式輔助

(Python 導入)



以腳本批次處理，效率大幅提升，
看見自動化的曙光。

 源自 Hady Tang 《Python 進階課程》

縮短至 2 週



3.0

我們的下一步

智能躍遷

(AI Agent 全自動化)



改以 AI Agent 進行開發，0 人工介入，
讓科技接手所有「髒活與累活」。

0 人工介入

我們的構想：延續這條從勞力到智力的進化，改用 AI Agent 開發——把人從重複勞動中釋放，讓「聽經」回到簡單而有尊嚴。



第一章

實用價值暨實際 DEMO 體驗



01

現有佛經資源，對特殊需求族群極不友善



視障者

既有 App 依賴視覺操作，即使搭配螢幕報讀仍冗長；有聲音檔無法語音控速、跳段，體驗破碎。



聽障者

誦經資源以「聽」為主，缺乏精準同步高亮文字的工具，看不到對應經文。



長輩

點小按鈕、打字、滑畫面門檻高，真正需要「像跟人說話」的自然互動。



共同缺口

多數 App 把無障礙當附加功能，少有「全語音設計優先」且兼顧視聽障雙模式者。



市場機會

台灣佛教人口眾多、長者與視聽障共修者比例高，卻無專屬語音佛經助手——明確服務缺口。

一款 App，四種人都受惠

對象	痛點	提供的價值
視障使用者	看不到螢幕	全語音對話 + 每步操作都有語音回饋
聽障使用者	跟不上誦經	逐句同步高亮、可放大字、可調語速
長輩	不會操作	開口說「念《心經》」就播，免點按
佛教團體	缺共修輔具	免費、合法、離線可用，適合道場推廣



「開口就能聽佛經，看著螢幕就能讀佛經
——讓視障與聽障使用者都能無障礙親近佛法的語音助手。」

對話式語音 Agent × 雙模式輸出



01 語音輸入

喚醒詞「小佛」、短時聆聽 → STT 轉文字，支援「Hey Google」，打開佛經助手」。



02 指令理解

以關鍵字 + 模糊比對辨識意圖，支援多種說法對應同一需求。



03 經文檢索

採用 CBETA 合法經文；首版收錄 3-5 部常見佛經，本地 SQLite + 雲端 Firestore 同步。



04 回應生成

整理對應經文與操作結果，提供即時、可理解的回覆內容。



05 雙模式輸出

TTS 即時語音回饋（可離線）+ 螢幕字幕逐句高亮，每次操作皆同步回報結果。



系統整合

背景播放、來電 / 拔耳機自動暫停、藍牙線控、續播記憶。



技術基底

Flutter + Dart 跨平台開發，離線優先，月固定成本趨近於零。

三個真實使用情境



情境一

視障長輩晨課

「Hey Google，打開佛經助手」
→ 「從頭念心經」→ 中途「慢一點」→ App 回「速度 0.8 倍」→
「重複本段」。全程不看螢幕完成。



情境二

聽障使用者跟讀

打開 App 選《心經》→ 字體放大、逐句同步高亮 → 調慢速度跟著默念 → 螢幕同步顯示助手文字回應。



情境三

中斷後續播

聽到一半來電 → App 自動暫停
→ 通話結束說「從剛才繼續」→
從中斷位置接續播放。

喚醒 → 播放 → 控制



為什麼是我們不一樣

比較項目	一般佛經 App	YouTube 音檔	有聲書 / CD	本專案
全語音操作	×	×	×	✓
視覺同步高亮	部分	×	×	✓
離線可用	部分	×	✓	✓
語音控速度 / 位置	×	×	×	✓
續播記憶	部分	×	×	✓
免費合法	部分	視頻道	×	✓

量化效益指標

看得見的改善



完成「播放指定佛經」時間

5 秒內

原本 30–60 秒 →



操作步驟數

1 句語音指令

原本 4–6 步點按 →



長輩首次上手教學

< 3 分鐘

首次上手 →

※ 以上為預估值，將於 DEMO 後實測驗證。

實作痛點與突破：攻克兩大開發難題

把「佛教破音字」與「大文本當機」兩大難關，轉化為差異化優勢

痛點 ①

佛教破音字，TTS 讀錯音



傳統 TTS 對佛教術語辨識率極低，常讀錯音，嚴重破壞聽經的專注與莊嚴。

× 般若 → ㄅㄢ ㄆㄛ ㄇㄣˊ ㄉㄜˋ

◆ 我們的突破



禪音獨家校正引擎：自建專屬詞庫字典，強制校正發音，並做智慧語氣優化還原自然語音。

般若 → ㄅㄢ ㄆㄛ ㄇㄣˊ ㄉㄜˋ

南無 → ㄋㄢˊ ㄩˊ ㄋㄢˊ ㄉㄜˋ

痛點 ②

大文本轉檔，記憶體溢位當機



佛經動輒數萬字，傳統轉檔工具極易記憶體溢位，導致系統崩潰當機。

× 數萬字 → 記憶體溢位 → 當機

◆ 我們的突破



大文本自動切檔：偵測超過 3,000 字自動分流，多工平行處理 (Multiprocessing) 速度翻倍。

3,000 字自動分流

速度 ×2

零當機

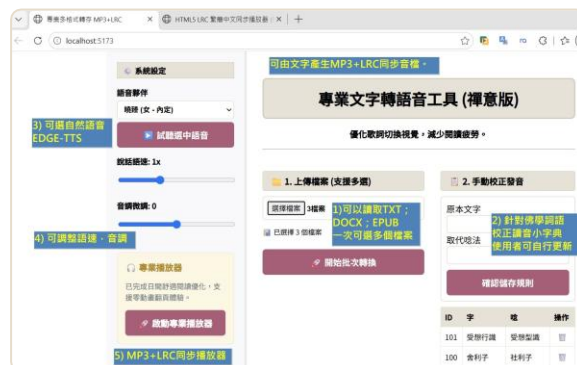
實機展示 | 三大工具，已實際運作

不只是構想——查經、轉語音、同步朗讀，全鏈路已實際跑通。



1 CBETA 多大腦查詢引擎

語意搜尋機器人 × API 書目檢索 × 全文檢索 × 地圖 Agent 總結導讀



2 文字轉語音 (禪意版)

TXT / DOCX / EPUB 批次轉檔 × 破音字校正字典 × EDGE-TTS 自然語音



3 MP3 + LRC 同步播放器

逐句高亮 × 變速播放 × 滑鼠滑動快速切句

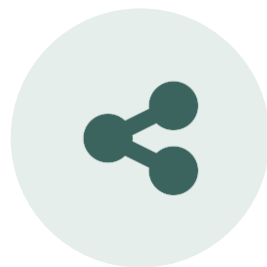
三大工具串起完整使用者旅程：查經 → 轉語音 → 同步朗讀

不只是一個 App



無障礙設計示範

可作為長照、教育、其他宗教類 App 的無障礙參考案例。



數位佛典推廣

讓 CBETA 資源觸及過去難以使用的族群。



可複製性

同架構可延伸至聖經、道教經典、長輩聽書等場景。

怎麼證明它真的有用

1

第一階段 (MVP 後)

邀 1-2 位視障 / 聽障朋友試用 10 分鐘，全程錄影記錄流程。

2

第二階段

與在地佛教共修團體合作辦小型 workshop，收集 5-10 位長輩回饋。

3

驗證重點

操作直覺性、語音辨識率、能否獨立完成核心任務（免協助）。



第二章

導入或商業化計劃

5 週做出可實測 MVP | 2 人小團隊 | 公司 token 成本 = 0

02

5 週完成 MVP

週次	重點工作	產出
W1	環境建置、CBETA 經文整理、App 骨架	空殼 App + 經文資料庫
W2	TTS 念經、句子高亮、播放控制、背景播放	可控制的播放器
W3	語音輸入、指令引擎、音量控制、狀態機	純語音操作播放器
W4	歷史紀錄、雲端同步、無障礙、雙模式	有記憶可同步的 App
W5	喚醒詞、狀態機微調、真實使用者實測	可試用的 MVP

先做核心，風險項目彈性處理

優先確保

W1-W4 核心功能

語音念經

語音控制（暫停 / 變速 / 音量）

從上次繼續（續播）

→ 必成項目，優先確保穩定可用

彈性項目

W5 高風險項目

喚醒詞（Porcupine）

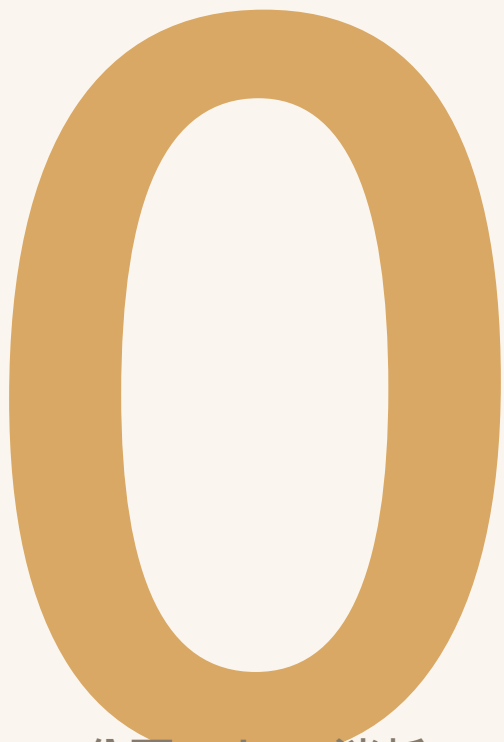
持續語音狀態機（最難）

TTS 與 STT 切換易互相干擾

→ 做不完就快速跟進，不延長總時程

成本估算

公司 token 成本 = 0



公司 token 消耗

語音助手實際運行全流程

LLM 指令理解、語音輸入輸出、喚醒詞、雲端同步，公司 token 消耗皆為 0。

LLM 走 Google AI Studio 免費額度

指令理解使用免費額度，公司 token 成本維持為零。

月固定成本趨近於 0

全部採用免費資源，無持續性付費項目。

每一項都是免費資源

成本項目	公司成本	說明
LLM 指令理解	無	Google AI Studio 免費額度
TTS 語音輸出	無	Android 內建，可離線
STT 語音輸入	無	系統內建，基本免費
喚醒詞 Porcupine	無	個人版免費
雲端 Firebase	無	免費 tier 足夠
佛經文字 CBETA	無	免費且合法

投入精簡、產出明確

2

人

共同開發

需求、撰寫、審查、測試

150–200

人時

5 週合計

每人每週 15–20 小時

1–2

位

使用者實測

第 5 週視障/聽障者協助 (志願)

每週一個可展示的檢核點

週末	驗收標準
W1	能載入經文、TTS 念出第一句 (開發者測試介面)
W2	完整念經、控制、句子高亮、背景播放正常 (按鈕為臨時測試)
W3	不碰螢幕純語音操作，皆有語音回報 (按鈕退場)
W4	「從上次繼續」、跨裝置歷史同步
W5	喚醒詞喚起，使用者 10 分鐘全語音完成全流程

風險已預先盤點

風險	因應對策
喚醒詞/狀態機最難 (TTS 與 STT 互擾)	列為彈性項目，先用按鈕喚起保底
內建 TTS 念經斷句不自然	有網切 Cloud TTS，先處理斷句
5 週緊湊、Flutter 不熟	每週驗收、善用 AI Agent 加速
免費額度上限、CBETA 授權	監控用量、標註出處

從小社群穩健擴散

1

封閉內測

先發給佛教團體少數成員試用，於內部測試管道驗證。

2

長輩友善上手

提供口頭/影片操作教學，設種子使用者協助旁人。

3

分批擴散

1-2 人 → 共修小組 → 整個團體，每階段收回饋再放大。

導入推廣才量得到的數字

試用完成率

受邀者中，完成 10 分鐘完整試用流程的比例。

獨立完成率

無旁人協助下，獨立完成「選經→播放→控制→續播」的比例。

種子擴散數

由種子使用者帶動上手的新使用者人數。

團體採用數

願意在共修場合實際使用 App 的團體 / 共修小組數。

長期維持零成本，逐步擴充

維運成本

token 與月固定成本持續趨近 0 (LLM 續用免費額度) 。

資金來源

公益性質，由團體支持或申請數位公益補助，非走營利模式。

後續發展路線

開放式問答

「《心經》在講什麼」
(沿用免費額度)



正式上架

Google Play
一次性約 US\$25



功能延伸

多語系、定時關閉、
重複播放

03

第三章

技術實現與工具運用

系統架構 | 處理流程 | 狀態機設計 | 技術堆疊

系統架構 — 六層分工

跨平台框架

Flutter + Dart 開發 · Android 優先

使用者互動層

語音輸入 | TTS 語音輸出 | 螢幕逐句高亮

語音感知與喚醒層

喚醒詞偵測 (小佛) | STT 語音辨識

Agent 核心層

LLM 指令理解 | 對話狀態機

播放與系統整合層

TTS 朗讀 | 背景播放 | 音量控制 | 系統事件

資料與內容層

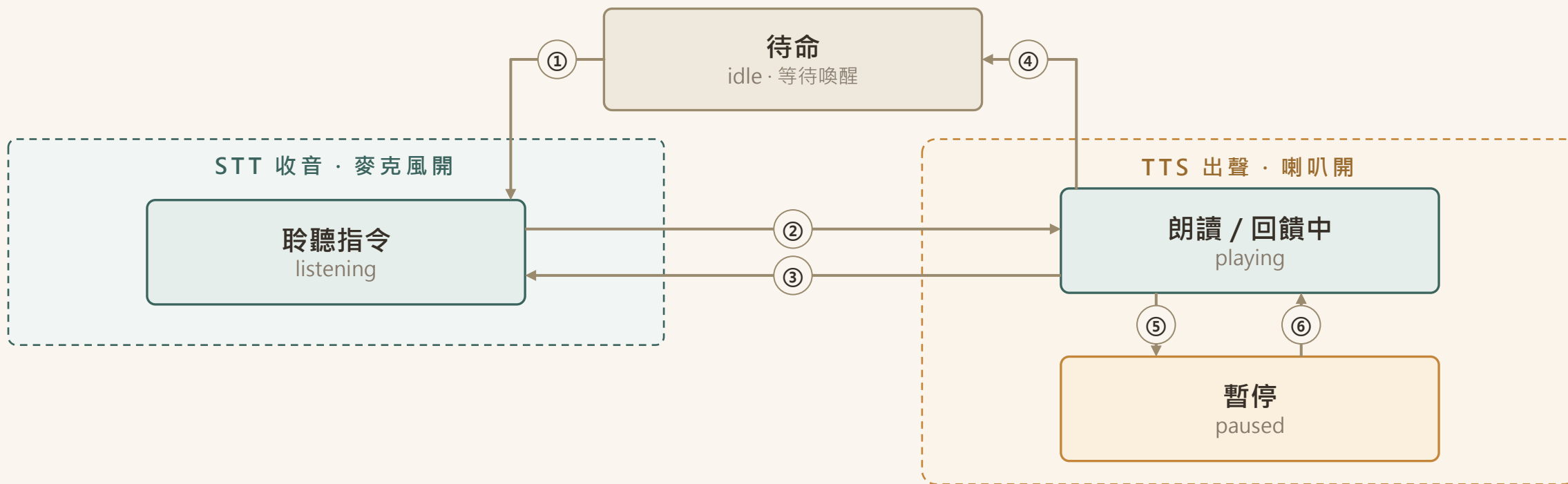
CBETA 經文 | SQLite 本地 | Firestore 雲端 | Firebase Auth

系統流程 — 一句語音指令的旅程



🔄 流程循環：回到待命後，持續等待下一個喚醒詞，逾時自動省電待命。

對話狀態機 — TTS 與 STT 互斥



① 喚醒詞觸發

④ 播放完畢

② 開始朗讀 / 回報

⑤ 來電 / 拔耳機 / 暫停

③ 再次喚醒，暫停 TTS

⑥ 從中斷處繼續

關鍵設計：只要 TTS 在出聲（念經或語音回報）就必須暫停 STT，避免麥克風收到 App 自己的聲音造成誤觸發。

技術堆疊 — 全部走免費資源

類別	功能用途	採用方案	備註
框架	App 開發框架	Flutter + Dart	跨平台、新手友善，iOS 不需重寫
語音	TTS 語音輸出	flutter_tts	系統內建可離線，有網路切 Cloud TTS
語音	STT 語音輸入	speech_to_text	系統 STT，基本免費
語音	喚醒詞偵測	Picovoice Porcupine	個人版免費，喚醒詞「小佛」
核心	指令理解 (NLU)	LLM · Google AI Studio	免費額度，公司 token 成本為零
播放	音量 / 背景播放	flutter_volume_controller · audio_service	控制系統媒體音量，鎖屏續播
整合	喚起 App	Google Assistant App Actions	「Hey Google」喚起
資料	本地 / 雲端 / 帳號	SQLite · Firestore · Firebase Auth	離線可用 + 跨裝置同步，免費 tier
內容	佛經文字來源	CBETA 中華電子佛典協會	免費、合法
第二版	開放式問答	Gemini Flash 免費 tier	升級問答能力，仍用免費額度

月固定成本趨近於 0:所有元件皆使用免費 tier,公司 token 成本維持為零。

04

第四章

資訊安全

隱私設計優先 | 資料最小化 | 儲存與存取保護 | 法規遵循

隱私設計優先 — 從源頭減少敏感資料

不儲存音檔

採 TTS 即時合成念經，使用者語音也只即時辨識、不錄存 —— 根本不產生音檔，自然消除外洩風險。

非持續錄音

喚醒詞「小佛」+ 短時聆聽模式，麥克風僅在被喚醒後短暫開啟，而非全程監聽，縮小隱私暴露面。

喚醒詞端側偵測

Porcupine 屬 on-device 引擎，喚醒詞的聲音完全不離開手機、不經雲端。

語音用完即丟

STT 將語音轉文字後即交付後續流程，系統不保留原始語音檔。

送 LLM 內容最小化

只把當下那句指令文字交給 LLM 解析，不夾帶帳號、播放歷史或經文內容，外送資料壓到最低。

儲存、傳輸與存取 — 平台級防護已就位

資料儲存與傳輸保護

- 離線優先，資料留在裝置

預設用系統內建 TTS/STT、經文存本地；斷網即可運作，資料不離手機。

- 本地 SQLite 儲存

經文與每部佛經最後播放位置存於本地，離線完成核心功能。

- 傳輸全程 TLS 加密

對外連線 (Firebase、Cloud TTS、AI Studio) 走 HTTPS，不以明文傳輸。

- 雲端靜態加密

Firestore 歷史同步享 Google Cloud 平台級靜態加密。

存取控制與系統穩定

- 委派式身份驗證

採 Firebase Auth (Google 登入)，App 端不自行保管任何密碼，降低帳號被竊風險。

- 使用免費合法內容

經文取自 CBETA 中華電子佛典協會，合法授權，避免侵權與來源不明風險。

- 狀態機防麥克風誤收

TTS 出聲時即暫停 STT，避免麥克風收到 App 自身聲音造成誤觸發，維持語音操作可控穩定。

結

總 結

回顧與展望

實用價值 | 導入商業化 | 技術實現 | 資訊安全

四個章節 — 一句話帶走重點

01 實用價值與 DEMO

為視障、聽障與長輩設計的全語音佛經助手：開口就播、逐句高亮、續播記憶，把播放一部經從 30–60 秒縮到 5 秒內。

02 導入與商業化

2 人小團隊、5 週做出可實測 MVP，從佛教社群封閉內測穩健擴散；公益定位，月固定成本趨近於 0。

03 技術實現與工具

Flutter 六層架構、TTS/STT 對話狀態機、離線優先；全棧採免費資源，公司 token 成本維持為零。

04 資訊安全

隱私設計優先：不存音檔、非持續錄音、喚醒詞端側偵測、送 LLM 內容最小化，加密與存取控制就位。

ZenVoice AI 禪音智友 | 無障礙語音 Agent 服務閉環

從使用者價值、核心流程、技術基底到資安設計，一頁看懂整體方案



開口能聽佛經，看著螢幕能讀佛經



喚醒



聆聽



理解



播放控制



雙模式回饋

01 使用者價值



視障者



聽障者



長輩



佛教團體

02 技術與資料基底



Flutter



對話狀態機



STT · TTS



LLM 理解



CBETA



SQLite · 雲端

03 導入與推廣



5 週 MVP



2 人團隊



內測擴散



月成本 ≈ 0

04 資訊安全與隱私



不存音檔



非持續錄音



端側偵測



LLM 最小化



TLS · Auth



結語

讓看不到的人聽見佛法，讓聽不到的人讀見佛法。

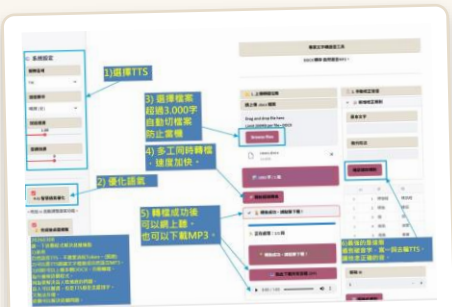
用最低的成本、最開放的技術，把數位佛典送到過去難以親近它的人手中。

這不只是一個 App，而是一個可被複製的無障礙公益範式。

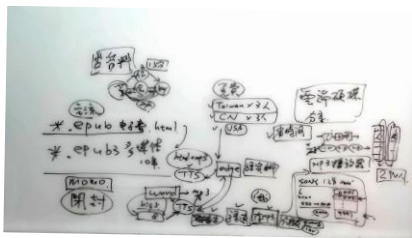
ZenVoice AI 禪音智友 | 佛教 × 無障礙公益

從白板到方案 | 兩人小組的討論歷程

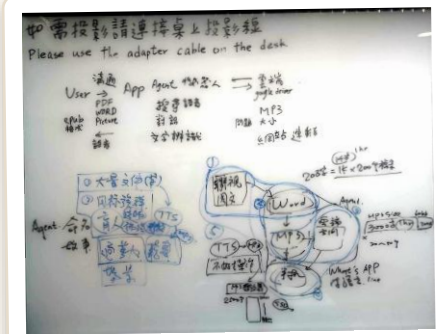
從問題與資源出發，經多輪白板共創，逐步收斂為可執行的系統方案。



01 文字轉語音原型



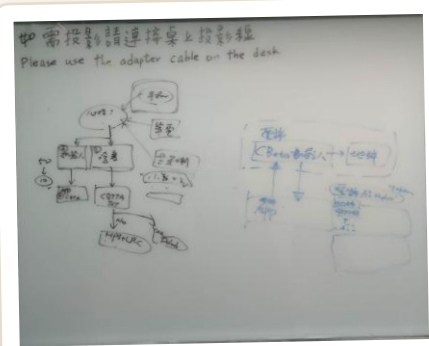
02 需求與資源盤點



03 流程與功能設計



04 技術管線與落地



05 近期流程架構

從最初的文字轉語音原型工具出發，歷經需求盤點、流程與功能設計、技術落地的多輪白板共創，逐步收斂為近期的系統流程架構，由兩人小組反覆討論而成。